Automated Library Synthesis in Homotopy Type Theory via Efficiency-Bounded Search

Halvor Lande

March 2026

Abstract

Automated reasoning systems usually optimize against a target supplied by the user: a theorem to prove, a conjecture to refute, or a type to inhabit. This draft studies a different problem. We ask whether an executable system can start from an empty type-theoretic library and decide which library extension should be added next under a fixed structural objective. The resulting system, PEN, operates over typed anonymous telescopes in a Minimal Binary Type Theory (MBTT) grammar, scores candidates by a target-independent efficiency ratio $\rho = \nu/\kappa$, and accepts the least supercritical candidate above a history-dependent bar.

The artifact is a working Haskell engine, supported by Cubical Agda mechanization and replay scripts. In the repository state studied here, the engine contains 50 Haskell modules and 19,517 lines under `engine/src`. The benchmark artifacts report a 15-step trajectory from an empty library to a temporal-cohesive endpoint, with structural discovery totals $\nu = 359$, $\kappa = 64$, a coherence-window sweep in which $d = 2$ outperforms $d = 1$ and $d = 3$, and an acceptance suite passing 46/46 tests. The same repository also contains stricter MBTT-first replay traces with raw and canonical candidate counts, near-miss diagnostics, and replay guards intended for CI.

The main JAR contribution is not a new foundational theorem alone, but an executable automated-reasoning system with a clear input/output contract: unlike QuickSpec, Hipster, IsaCoSy, or Synquid, PEN does not receive a fixed conjecture, signature-completion task, or refinement type. It receives a library state and must decide which theory fragment to build next. At the same time, the draft is explicit about current engineering limits. The evaluator is structural and deterministic, but the present search layer remains guided for tractability through goal profiles, agenda search, canonical quotienting, bounded rollouts, and phase-specific ranking penalties. We therefore separate a strong claim about the scoring objective from a narrower implemented finding about the current claim-grade search lane.

1 Introduction

Most successful automated-reasoning systems optimize a *local* target. In theorem proving, the target is a conjecture or proof state; in theory exploration, it is a fixed signature whose latent equations or lemmas are to be surfaced; in type-directed synthesis, it is a requested type or refinement contract. PEN asks a different question: if the target is not a theorem but the *growth of the library itself*, what should an automated reasoner do next?

This paper presents a first journal-style account of PEN (Principle of Efficient Novelty) as an executable automated-reasoning system for library synthesis in intensional Homotopy Type Theory. Starting from an empty library, the engine generates typed candidate extensions, computes a fixed structural score for each candidate, and inserts the candidate whose efficiency

$$\rho(X) = \nu(X)/\kappa(X)$$

clears the current survival bar with minimal positive overshoot. Here ν is the candidate's marginal generative capacity and κ is irreducible construction effort. The bar itself is history-dependent: it is driven by previously realized novelty and by a coherence-window recurrence that raises the cost of integration as the library grows.

The central empirical finding is that a reproducible executable system recovers a 15-step sequence of library extensions under this objective. In the repository snapshot available here, the benchmark harness reports:

- a structural discovery replay with all 15 canonical names in order, total $\nu = 359$, total $\kappa = 64$;
- a paper-calibrated replay with the same 15-step order and total $\nu = 356$;
- a coherence-window sweep where the $d = 2$ lane strictly dominates the $d = 1$ and $d = 3$ lanes under the benchmark metric;
- an acceptance suite passing 46/46 tests.

The trajectory itself is geometric: universe, unit, witness, dependent type formers, S^1, truncation, S^2, S^3, Hopf, cohesion, connections, curvature, metric, Hilbert, and a temporal-cohesive endpoint interpreted as DCT.

Why this is a different automated-reasoning problem. The cleanest way to position PEN is through its input/output contract. Table 1 summarizes the distinction.

Table 1: PEN contrasted with nearby automated-reasoning families.

Family	Input	Search object	Main acceptance signal	Output
Goal-conditioned proving [7, 8, 9]	Conjecture or proof state	Proof steps / proof states	Proves the given goal	Proof object or proof script
Theory exploration [10, 11, 12]	Fixed signature or theory	Equations, lemmas, conjectures	Interestingness / provability inside a fixed theory	Lemma set, equations, conjectures
Type-directed synthesis [13]	Type or refinement type	Programs inhabiting that target	Satisfies requested type and local objective	Program term
PEN	Current library state and fixed objective	Typed library extensions	Clears a structural bar under $\rho = \nu/\kappa$	Next library entry and a trajectory of theory growth

QuickSpec, Hipster, and IsaCoSy are especially close conceptually, because they also operate without a single externally supplied theorem. But their background theory is fixed. They ask which *facts* should be discovered inside a theory already chosen by the user. PEN instead asks which *theory fragment* should be added next. That shift matters: PEN's search objects are typed APIs and anonymous MBTT telescopes, not conjectures. Its objective is not explanatory compression of an existing algebra, but growth of the library under a fixed structural utility.

Claim boundary. The strongest safe claim is about the evaluator, not the entire search stack. The evaluator used by PEN is target-independent in the narrow sense that ν, κ, and ρ are computed structurally from the candidate and library state; the claim-grade source guard in `RunAbInitio.hs` explicitly rejects `REF` and `PAPER` sources in strict and structural lanes. However, the current search-control layer is *not* free of guidance. It uses goal profiles, agenda search, canonical quotienting, bounded MCTS, and late-phase ranking penalties keyed to structurally detected names. The right framing is therefore:

- **Objective-level claim:** PEN evaluates library extensions under a fixed structural objective rather than target-conditioned theorem scoring.
- **Implemented finding:** the current repository recovers the 15-step sequence in a guided, reproducible claim-grade lane.

That distinction is central to the rest of the paper.

Contributions. This draft makes five contributions aimed at a JAR audience:

1. It reframes PEN as a new automated-reasoning task: theory growth as optimization target.
2. It specifies the executable model in system terms: typed candidates, dual costs, admissibility, bar dynamics, and deterministic selection.
3. It documents the engine architecture using the current codebase rather than only the mathematical narrative.
4. It reports the 15-step trajectory together with search telemetry, ablations, and rejected-candidate evidence.
5. It separates discovery, structural audit, uniform cross-checking, and Agda mechanization, making the remaining engineering gaps explicit instead of implicit.

2 PEN as an Executable Specification

This section keeps the mathematics only to the extent needed to understand what the engine computes and what invariants it preserves.

2.1 State, candidate, and score

Definition 2.1 (Library state). *A library state $\mathcal{B}_n$ is a monotone sequence of previously sealed entries. In the implementation this state is represented by a list of* `LibraryEntry` *values recording constructors, path dimensions, and capability flags needed by subsequent search.*

Definition 2.2 (Candidate). *A candidate X is a typed extension of the current library. Concretely, the engine represents candidates as anonymous MBTT telescopes together with derived metadata such as clause count, bit-length, canonical key, and post-hoc decode label.*

Definition 2.3 (Dual costs). *PEN uses two costs:*

1. ***Construction effort*** *$\kappa(X)$: the irreducible clause count of the chosen specification. The codebase also reports entry-count and MBTT bit-cost variants.*
2. ***Integration latency*** *Δ_n: the size of the coherence burden incurred when sealing the next entry against the recent interface.*

Definition 2.4 (Novelty and efficiency). *The novelty of a candidate is*

$$\nu(X \mid \mathcal{B}) = |\mathcal{L}(\mathcal{B} \cup \{X\})| - |\mathcal{L}(\mathcal{B})|,$$

and the efficiency score is $\rho(X) = \nu(X)/\kappa(X)$. In the repository there are multiple ways to instantiate ν:

- ***Structural / spectral mode:*** `StructuralNu.hs` *computes $\nu_G + \nu_H + \nu_C$ directly from the MBTT AST.*
- ***Uniform mode:*** `UniformNu.hs` *computes before/after differences in inhabited types and adds adjoint-completion credits.*
- ***Paper-calibrated replay:*** *legacy comparison lane used for regression and reproducibility against older tables.*

2.2 Bar dynamics

Let Ω_{n-1} be the cumulative realized novelty-to-effort ratio up to step $n-1$, and let $\Phi_n = \Delta_n/\Delta_{n-1}$ be the structural inflation term induced by the chosen coherence window. The selection bar is

$$\mathrm{Bar}_n = \Phi_n \cdot \Omega_{n-1}.$$

The executable implementation computes this quantity directly in `computeBarD`. For $d = 2$, the recurrence is Fibonacci-based through `dBonacciDelta`; for the first two steps the implementation fixes $\mathrm{Bar}_1 = \mathrm{Bar}_2 = 0.5$.

2.3 Five executable invariants

The original paper presents PEN axiomatically. For the system paper, the same content is more useful as executable contracts; see Table 2.

Table 2: From mathematical axioms to executable contracts.

Invariant	Executable reading
Monotone growth	The library only grows; selected entries are appended and never removed.
Bounded admissibility	A candidate must be well-formed, typed enough to pass the conservative checker, connected to the current library, and within search budgets.
Coherence-sensitive bar	Acceptance is relative to a history-dependent bar computed from prior realized novelty and the d-Bonacci latency sequence.
Minimal-overshoot selection	Among bar-clearers, the engine minimizes positive overshoot and then breaks ties deterministically by lower κ and stable structural keys.
Sealing and export	The winner becomes a new library entry and may be exported to reporting and Agda bridge formats, but reporting metadata must not interfere with selection.

The last item is enforced in several places in the repository. `MBTTDecode.hs` is explicitly reporting-only, and the Phase 5 reports add a non-interference guard. The Agda bridge similarly treats discovery and verification as separate surfaces.

2.4 Executable loop

At a high level the engine implements the following loop:

```
for step n:
  C0 <- generate typed candidates from the current library
  C1 <- prune C0 by local type checks and structural filters
  C2 <- quotient C1 by canonical MBTT key
  E  <- evaluate each candidate in the selected nu/kappa mode
  V  <- { x in E | rho(x) >= Bar_n }
  if V is empty: stop or idle
  else:
    choose x* with minimal positive overshoot
    seal x* into the library
```

This description is intentionally close to the implementation in `RunAbInitio.hs`. The important point is that the acceptance logic is simple and global: the candidate set can be generated in different ways, but acceptance is always phrased against the same bar and the same fixed structural utility.

3 Engine Architecture

3.1 Repository-scale view

The current repository is not a toy prototype. The architecture document and line-count audit report:

- 50 Haskell modules under `engine/src`;
- 19,517 Haskell lines in `engine/src`;

- 19,735 Haskell lines repository-wide, excluding build artifacts;
- a parallel Cubical Agda mechanization under `agda/`;
- replay, benchmark, and evidence scripts under `scripts/` and `engine/scripts/`.

Table 3 gives the architecture at the level useful to a JAR reader.

Table 3: Main architectural groups in the current codebase.

Group	Representative modules	Role
Search-space generation	`MBTTEnum`, `AgendaSearch`, `Generator`, `MCTS`	Enumerate or search typed candidate telescopes under budgets and goal profiles.
Checking and canonicalization	`TelescopeCheck`, `MBTTCanonical`, `Telescope`	Conservative well-formedness checks, canonical keys, telescope utilities, and structural classification.
Scoring and selection	`StructuralNu`, `UniformNu`, `TelescopeEval`, `RunAbInitio`	Compute ν, κ, ρ, the bar, and the final winner.
Search guidance	`ProofState`, `ProofRank`, `CoherenceWindow`	Goal profiles, debt graphs, capability signatures, premise buckets, and coherence-derived budgets.
Reporting and bridge	`MBTTDecode`, `AgdaExport`	Post-hoc semantic labels, bridge payloads, Agda stubs, and non-interfering reporting.

3.2 Candidate generation in the current engine

The current codebase is more informative than the older conference manuscript on one point: the implementation has already migrated toward MBTT-first search. The default runtime no longer follows the earlier split "combinatorial steps 1–9, guided steps 10–15" in a literal sense. Instead:

- steps 1–3 in the available strict full replay are obtained by direct MBTT enumeration (`ENUM_MBTT`);
- steps 4–15 are obtained by agenda-guided search over typed MBTT telescopes (`AGENDA`);
- a legacy template-first lane remains present behind `-legacy-generator` as rollback support rather than as the primary research path.

That distinction matters for the JAR paper because it shifts the engineering narrative from "category templates that happen to recover the intended sequence" to "a typed anonymous search space with several search-control policies."

Enumeration. `MBTTEnum.hs` enumerates expressions and telescopes under explicit bit, depth, and candidate bounds. It emits anonymous candidates annotated with:

- MBTT bit-length;
- desugared clause count;
- AST node count;
- a heuristic relevance score derived from the current goal profile.

Enumeration is typed only in a lightweight sense: the system generates terms inside an MBTT grammar and then relies on a conservative telescope checker plus downstream structural tests.

Obligation-driven agenda search. The most distinctive current component is `AgendaSearch.hs`. Search is carried not only by syntax but by an explicit obligation state:

- `SearchState` tracks capability signatures, open obligations, a debt graph, premise buckets, and lower/upper novelty bounds.
- `GoalProfile` and `GoalIntent` determine which obligations matter at the current step.
- critic plans and macro templates rewrite or extend states before expensive branching.

- dominance caches and sigma-upper-bound pruning discard low-potential states.

This is the main tractability device for late-phase synthesis. In a full strict 15-step replay (`engine/r15_temporal_steps.csv` and companion prefix log), the late steps are reached with very small viable frontiers because the agenda mechanism has already compressed the search surface heavily.

Type checking and pruning. `TelescopeCheck.hs` is conservative rather than complete: it aims to reject obviously malformed telescopes while avoiding false negatives on well-formed ones. The checker enforces library-reference bounds, variable-scope bounds, binder discipline, and a few term-level sanity constraints such as banning `App _ Univ`. Additional filters require connectedness to the existing library and coherence-window compatibility.

Canonical quotienting. `MBTTCanonical.hs` provides the current canonicalization contract. The present version is intentionally modest:

- recursive structural normalization only;
- deterministic canonical keys by hashed canonical encodings;
- no beta/eta reduction, no alpha-normalization, no commutativity quotient.

This matters because the JAR story is not "we solved equivalence of MBTT telescopes." It is "we implemented a deterministic first quotient strong enough to reduce duplicate search paths without destabilizing replay." The Phase 2 differential report records a 48.33% candidate reduction on a bounded differential gate, exceeding the repository's 40% threshold.

Selection and representative choice. Selection happens twice. First, equivalent candidates are quotiented by canonical key; when two candidates share a key, the representative choice prefers lower κ, then higher ρ, then deterministic source rank. Second, after bar gating, the surviving candidates are ranked by overshoot plus phase-specific penalties, then by lower κ, then by structural surplus and source rank. This is the point where the implementation is still guided: late-phase penalties explicitly prefer structurally recognized connections, cohesion, curvature, metric, Hilbert, and DCT bundles when such candidates are already viable.

MCTS fallback. `MCTS.hs` treats synthesis as a finite-horizon MDP over partial telescopes. It uses UCT selection, progressive widening, random rollouts, and backpropagation. In the code path of `RunAbInitio.hs`, MCTS is a *fallback or portfolio* lane rather than the default mechanism: it is invoked when the estimated relevant κ exceeds the enumeration horizon or when enumeration returns no viable candidates. In the available full strict replay used below, MCTS is not exercised, but the architecture is present and the paper should document it because it is part of the intended scaling story.

3.3 Search telemetry and budgets

The repository exposes search budgets directly in run logs. The strict prefix alignment logs show:

- adaptive memory-aware budgets for enumeration and MCTS;
- peak memory around 1 GiB during early search in one replay log;
- CI-oriented shadow gates requiring bounded replay through step 7 within a 30-second wall-clock envelope on provisioned runners;
- richer per-step diagnostics such as dominance-prune counts, sigma-prune counts, action failures, and near misses.

This is worth emphasizing because the system is closer to a search engine with artifact contracts than to a one-off research script.

4 Results

4.1 The 15-step trajectory

Table 4 records the main structural-discovery trajectory from the baseline benchmark artifact `runs/phase0_baseline/structural_d2.csv`. These are the numbers most consistent with the current mathematical exposition in `pen_paper.tex`: the late-stage structural values are $\nu = 46, 62, 103$ for Metric, Hilbert, and DCT respectively.

Two points are worth stressing for a JAR audience.

First, the sequence is not being read off from a human-curated list at runtime. The current strict and structural lanes explicitly reject `REF` and `PAPER` candidate sources as claim-grade inputs. The post-hoc names in the table are reporting labels attached after structural selection.

Second, PEN is *not* a max-ρ optimizer. Its selection rule is minimal overshoot above the bar. In the strict 15-step replay used for telemetry, step 4 selects Pi with $\rho = 1.6667$ over a competing candidate with $\rho = 1.75$ because Pi's overshoot above the bar 1.50 is smaller; step 14 similarly selects Hilbert with $\rho = 6.8889$ over a competitor at $\rho = 7.1111$ because the former is the closer bar-clearer. These are exactly the kinds of decisions that distinguish library growth from raw greedy reward maximization.

4.2 Search-space telemetry

The older phase-0 structural CSVs report only total candidate counts per step. The current repository also includes richer strict MBTT-first telemetry with raw candidates, canonical survivors, viable frontiers, and agenda diagnostics. Table 5 summarizes a full 15-step strict replay from `engine/r15_temporal_steps.csv` and `engine/r15_temporal_prefix.csv`.

This table makes three engineering points concrete.

1. The search surface is not remotely the raw MBTT bitstring space. Even in the strict 15-step replay, the total number of raw candidates examined across the winning steps is only 123 because the search is already heavily structured by typing, agenda obligations, and pruning.
2. Canonical quotienting matters early. Step 6 contracts from 37 raw candidates to 9 canonical survivors, a 75.7% reduction. This is consistent with the repository's separate phase-2 differential report, which records a 48.33% reduction in a bounded differential gate.
3. Late-stage synthesis is narrow. Steps 10–15 all finish with raw frontiers of size 1–3. That is not a proof of uniqueness in the full admissible space, but it is a strong empirical indication that the guided lane is no longer exploring a broad uncontrolled frontier by the time it reaches cohesive, differential, and temporal structure.

Agenda diagnostics. The prefix reports also expose state-expansion and pruning counts. In the same strict replay, step 15 expands 140 agenda states and prunes 740 by dominance; step 12 prunes 296 states by sigma upper bounds. These counts are important because they show where the current implementation is getting its tractability from: the frontier is small *after* aggressive obligation-guided pruning, not before it.

4.3 Rejected candidates and ablations

The current conference manuscript pushes rejected candidates to an appendix. For a JAR paper they deserve more prominence because they show that the engine is selective rather than scripted.

Table 4: Structural-discovery trajectory from the phase-0 benchmark artifact (`d=2` lane).

Step	Structure	ν	κ	ρ	Bar	Δ_n
1	Universe	1	2	0.50	0.50	1
2	Unit	1	1	1.00	0.50	1
3	Witness	2	1	2.00	1.33	2
4	Pi	5	3	1.67	1.50	3
5	S1	7	3	2.33	2.14	5
6	Trunc	8	3	2.67	2.56	8
7	S2	10	3	3.33	3.00	13
8	S3	18	5	3.60	3.43	21
9	Hopf	17	4	4.25	4.01	34
10	Cohesion	19	4	4.75	4.46	55
11	Connections	26	5	5.20	4.91	89
12	Curvature	34	6	5.67	5.43	144
13	Metric	46	7	6.57	5.99	233
14	Hilbert	62	9	6.89	6.68	377
15	DCT	103	8	12.88	7.40	610

Table 5: Per-step frontier telemetry from a full strict MBTT-first replay. Totals across the 15 steps are 123 raw candidates and 77 canonical/viable candidates; average dedupe ratio is 0.8184.

Step	Structure	Raw	Canonical	Viable	Source
1	Universe	12	11	11	ENUM_MBTT
2	Unit	11	11	11	ENUM_MBTT
3	Witness	7	3	3	ENUM_MBTT
4	Pi	32	22	22	AGENDA
5	S1	4	2	2	AGENDA
6	Trunc	37	9	9	AGENDA
7	S2	5	5	5	AGENDA
8	S3	2	2	2	AGENDA
9	Hopf	2	1	1	AGENDA
10	Cohesion	1	1	1	AGENDA
11	Connections	2	2	2	AGENDA
12	Curvature	2	2	2	AGENDA
13	Metric	1	1	1	AGENDA
14	Hilbert	3	3	3	AGENDA
15	DCT	2	2	2	AGENDA

Table 6: Representative rejected candidates, adapted from the existing manuscript and artifacts.

Candidate	ν	κ	ρ	Reason for rejection
Natural numbers $\mathbb{N}$	≤ 4	3	≤ 1.33	Fails geometric bar once higher-path structures become admissible
Classical logic (LEM)	≤ 1	1	≤ 1.00	Minimal novelty, no higher-path gain
Power-set style axiom	≤ 2	2	≤ 1.00	Too little inferential interaction per clause
Ordinal arithmetic	≤ 6	5	≤ 1.20	Discrete growth remains subcritical
Measure theory	≈ 12	6	≈ 2.0	Arrives too late with insufficient efficiency
Elementary topos axioms	≈ 15	8	≈ 1.9	High cost, weak local amplification in this objective
Lie groups	9	6	1.50	Loses to modal and differential packages
Scheme theory	≈ 18	7	≈ 2.6	Too discrete under the current score
Galois theory	≈ 10	5	≈ 2.0	No comparable homotopy contribution
Symplectic geometry	≈ 25	5	≈ 5.0	Closest geometric near miss, still below the step-13 bar

There are two different kinds of negative evidence in the repository.

Domain-level rejections. The table above shows families the manuscript already argues against on structural grounds. The unifying pattern is that discrete packages have little or no ν_H contribution and therefore cannot keep pace with a rising coherence-sensitive bar.

Search-policy ablations. The phase-0 baseline directory also contains two particularly useful negative controls.

- `ablation_no_canonical.csv`: disabling canonical priority quickly derails the intended sequence, already yielding unnamed or suspension-like candidates in the early trajectory.
- `ablation_max_rho.csv`: replacing minimal-overshoot selection by pure max-ρ produces degenerate one-clause fragments and destroys the intended cumulative dynamics almost immediately.

These ablations are valuable because they show that the core behavior is not robust to arbitrary selection objectives. The ordering depends on the specific PEN choice of bar gating plus minimal overshoot.

5 Verification and Cross-Validation

PEN's credibility does not come from a single number. It comes from a stack of partially independent checks that should agree on order even when they disagree on magnitude.

5.1 Structural, uniform, and audit lanes

Structural lane. The primary publication-grade benchmark artifact is the structural discovery run (`structural_d2.csv`), where ν comes from `StructuralNu.hs`. This lane yields the 15-step trajectory with total $\nu = 359$ and $\kappa = 64$.

Uniform lane. `UniformNu.hs` computes novelty by before/after type inhabitation comparison plus adjoint-completion credits. Its purpose is not to drive the search by default but to test whether the structural ordering survives a different measurement discipline.

Meta-theoretic audit. The manuscript also contains a rule-by-rule audit of ν_G, ν_C, and ν_H. For a JAR version this should be read as a mechanized explanatory layer over the structural AST computation rather than as a replacement for it.

5.2 Where the lanes agree and where they diverge

Table 7 summarizes the most important agreement and divergence points visible in the current repository.

Table 7: Replay and verification lanes in the current repository.

Lane	15-step order	Total ν	Comment
Paper-calibrated replay	Yes	356	Legacy comparison lane; late-step values are 43/60/105 for Metric/Hilbert/DCT.
Structural discovery ($d = 2$)	Yes	359	Primary structural baseline; late-step values are 46/62/103.
Strict MBTT-first replay	Yes	333	Same order in the available full replay, but DCT is scored at 77 rather than 103.
$d = 1$ structural sweep	Yes in benchmark artifact	347	Inferior to $d = 2$ under the benchmark total.
$d = 3$ structural sweep	Yes in benchmark artifact	195	Much weaker than $d = 2$ under the benchmark total.
Acceptance suite	N/A	N/A	46/46 passing in the phase-0 benchmark snapshot.

This table exposes two kinds of engineering debt that a JAR paper should not hide.

Late-step quantitative drift. The current repository no longer has a single authoritative late-step novelty table. The older paper-calibrated lane uses $(43, 60, 105)$ for steps 13–15; the structural lane uses $(46, 62, 103)$; and a full strict replay agrees through step 14 but assigns DCT only 77. The order is stable; the magnitudes are not. This should be framed as an engineering and measurement divergence, not as a reason to doubt that the system recovered the same structural sequence.

Window-sweep semantics. The narrative paper argues that $d = 1$ stagnates and $d = 3$ overconstrains growth. The current benchmark harness instead reports 15-step sweeps for all three windows, using total novelty as the comparative metric. The code and artifact claims are therefore narrower than the strongest mathematical prose. A journal version should state this explicitly.

5.3 Name independence, decode non-interference, and discovery/verification split

Several repository reports are directly relevant to the claim that the evaluator is not secretly target-conditioned.

- The Phase 3 name-independence report records a fix replacing name-gated temporal capability assignment with structure-driven detection of `Next` and `Eventually`, and reports structural/name-free behavior through step 15.
- The Phase 5 reports establish a reporting-only decoder boundary. `MBTTDecode.hs` supplies post-hoc labels and confidence values but is guarded to remain non-interfering with selection.
- The Phase 6 reports enforce a discovery/verification split: the Haskell engine proposes anonymous MBTT candidates and exports payloads, while Agda-side bridge contracts verify the shape of those claims on a separate surface.

This is one of the cleaner engineering stories in the repository. Even though the search layer is still guided, the codebase does contain explicit barriers against a simpler failure mode in which decoded semantic names leak back into the evaluator by accident.

6 Cubical Agda Mechanization

The Agda development is not a decorative appendix. It covers three distinct roles.

Recurrence and coherence mathematics. `ObligationGraph/Recurrence.agda`, `Core/Nat.agda`, and related modules formalize the Fibonacci-style recurrence story behind the $d = 2$ coherence window. This is where the draft's cost recurrence is most cleanly grounded.

Abstraction barriers and local obligations. The existing manuscript's strongest Agda material is the abstraction-barrier story around steps 8–9. The corresponding modules `Saturation/AbstractionBa` and `Saturation/AbstractionBarrier9.agda` demonstrate that obligations can be discharged from sealed interfaces rather than by illegitimate access to deeper history. That is exactly the kind of mechanized local invariant a JAR paper should highlight.

Bridge contracts and proof obligations. The Phase 6 bridge harness adds a distinct contribution: `agda/Test/BridgePayloadContract.agda` mirrors the discovery payload schema with records for `NuClaim` and `BridgePayload`, and postulates explicit obligations such as `CanonicalKeySound`, `NuClaimWellFormed`, and `DecodeNonInterference`. This does not yet prove the whole synthesis theorem in Agda, but it does put the cross-language contract on a formal footing.

For a journal paper, the right message is therefore not "Agda fully verifies the engine." It is more precise:

- recurrence facts are mechanized;
- abstraction-barrier obligations are mechanized for the key transition around steps 8–9;
- the bridge surface is formalized as an explicit verification harness;
- the full end-to-end search theorem remains outside the Agda mechanization.

7 Limitations and Guidance for a Full JAR Version

The repository is strong enough for a first journal draft, but several issues should be front-and-center rather than deferred.

7.1 Conjecture versus implemented finding

The manuscript should keep the following distinction explicit in the introduction, discussion, and conclusion.

- **Conjecture:** stronger, less guided MBTT-first search over the same admissible space would recover the same 15-step trajectory.
- **Implemented finding:** the current codebase recovers that trajectory in a guided claim-grade lane using agenda search, seed families, canonical quotienting, and late-phase ranking controls.

The journal draft should not collapse these into a single "autonomous discovery" claim.

7.2 Direct baselines against theory-exploration systems are still missing

The related-work contrast with QuickSpec, Hipster, IsaCoSy, and Synquid is already conceptually strong, but this first draft does not yet include an executable head-to-head experiment on overlapping toy domains. That absence should be stated plainly. A strong next revision would include:

- a small overlapping benchmark where QuickSpec/Hipster/IsaCoSy explore equations inside a fixed signature;
- the corresponding PEN run where the problem is not equation discovery but theory-extension choice;

- a short analysis showing that the systems are solving genuinely different tasks even when the surface domain is similar.

7.3 Current canonicalization is intentionally weak

The present quotient only collapses deterministic structural duplicates. It does not yet identify alpha-equivalent, beta-equivalent, or symmetry-equivalent telescopes. This is an acceptable engineering choice for reproducibility, but it limits how strongly one can talk about the "true" size of the admissible search space.

7.4 Quantitative drift needs one authoritative table

As documented in Table 7, the repository currently contains multiple late-step novelty calibrations. A full JAR submission should choose one table as authoritative, explain exactly how the others arose, and treat the rest as compatibility or regression artifacts.

7.5 Scalability claims should stay modest

The evidence contract is already better than many research artifacts: it includes replay scripts, acceptance suites, CI ladders, and shadow-prefix timing gates. But it is not yet the same thing as a broad runtime study. The paper should therefore avoid vague claims of scalability and instead state concrete current evidence:

- full 15-step replays exist in repository artifacts;
- bounded shadow replays through step 7 are CI-gated;
- structural replay remains significantly heavier than strict prefix replay;
- late-stage search is tractable in the current guided lane largely because the agenda mechanism makes the frontier narrow.

8 Conclusion

PEN is best understood as a working automated-reasoning system for *library synthesis*. Its novelty is not that it proves one more theorem in HoTT, but that it operationalizes a different target: which typed structure should be added to the library next under a fixed structural objective.

In the current repository state, that idea is backed by a real implementation, reproducible artifacts, a 15-step trajectory from an empty library, multiple verification lanes, and explicit engineering contracts around name independence and reporting non-interference. At the same time, the codebase also makes the remaining limitations visible: guided search control is still doing real work, direct experimental comparison with theory-exploration systems has not yet been run, and late-step novelty numbers still drift across replay lanes.

That combination is exactly what makes the project promising for JAR. There is already a working system and an unusually rich artifact trail. The right journal paper is therefore not a more expansive philosophical version of the conference manuscript. It is a more explicit systems paper: what the engine computes, how the search space is controlled, where the evidence is strongest, and which claims remain conjectural.

References

[1] Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, 2013.

[2] C. Cohen, T. Coquand, S. Huber, A. Mörtberg. Cubical Type Theory: a constructive interpretation of the univalence axiom. *TYPES 2015*, 2015.

[3] U. Schreiber. Differential Cohomology in a Cohesive Infinity-Topos. arXiv:1310.7930, 2013.

[4] F. W. Lawvere. Axiomatic Cohesion. *Theory and Applications of Categories*, 19(3), 2007.

[5] H. Nakano. A Modality for Recursion. *Proceedings of LICS*, 2000.

[6] A. Vezzosi, A. Mörtberg, A. Abel. Cubical Agda: A Dependently Typed Programming Language with Univalence and Higher Inductive Types. *ICFP*, 2019.

[7] T. Trinh, Y. Wu, Q. Le, H. He, and collaborators. Solving Olympiad Geometry without Human Demonstrations. *Nature*, 2024.

[8] K. Yang, A. Selsam, K. Pang, et al. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. *NeurIPS*, 2023.

[9] OpenAI. Advancing AI-Assisted Formal Mathematics. Research update, 2025. https://openai.com/index/advancing-ai-assisted-formal-mathematics/

[10] N. Smallbone, M. Johansson, K. Claessen, M. Algehed. Quick Specifications for the Busy Programmer. *Journal of Functional Programming*, 27:e18, 2017.

[11] M. Johansson, L. Dixon, A. Bundy. Conjecture Synthesis for Inductive Theories. *Journal of Automated Reasoning*, 47:251–289, 2011.

[12] M. Johansson, D. Rosén, N. Smallbone, K. Claessen. Hipster: Integrating Theory Exploration in a Proof Assistant. *CICM*, LNCS 8543, pp. 108–122, 2014.

[13] N. Polikarpova, I. Kuraj, A. Solar-Lezama. Program Synthesis from Polymorphic Refinement Types. *PLDI*, 2016.

[14] D. B. Lenat. *AM: Discovery in Mathematics as Heuristic Search.* McGraw-Hill, 1983.

[15] S. Colton. *Automated Theory Formation in Pure Mathematics.* Springer, 2002.

[16] M. Li, P. Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications.* Springer, 4th edition, 2019.